



PyQuest

Information- & Aufgabenblatt

Kapitel 16: Projekte

Bring alles zusammen, was du gelernt hast, in zwei kleinen Projekten.

Projekt: Zahlenraten

Zeit für dein erstes richtiges Projekt! Ein Programm folgt immer einer festen Abfolge von Schritten, die zu einem Ziel führt – so eine Abfolge nennt man **Algorithmus**. Du baust ein **Zahlenraten-Spiel**: Das Programm denkt sich eine Zahl aus (die wir hier fest vorgeben) und die spielende Person muss sie erraten. Nach jedem Versuch bekommt sie einen Hinweis: zu hoch oder zu niedrig.

Das kombiniert alles, was du schon kannst:

- **while True**: für eine Schleife, die läuft, bis sie mit **break** beendet wird
- **input()** und **int()** für die Eingabe
- **if/elif/else** für die drei möglichen Fälle
- **break**, um bei richtiger Antwort aufzuhören

Aufgabe 1: Warum eignet sich **while True**: zusammen mit **break** für dieses Spiel?

- ☐ a) Weil man vorher nicht weiß, wie viele Versuche gebraucht werden
- ☐ b) Weil **while True**: automatisch nach 3 Versuchen aufhört
- ☐ c) Weil **for**-Schleifen hier nicht erlaubt sind



Aufgabe 2: Die Geheimzahl ist bereits als `geheimzahl = 7` vorgegeben. Schreibe eine Schleife, die mit `input("Rate die Zahl: ")` wiederholt einen Versuch abfragt (als Zahl umgewandelt). Gib `Zu niedrig!` aus, wenn der Versuch kleiner ist, `Zu hoch!`, wenn er größer ist, und `Richtig!`, wenn er stimmt – dann soll die Schleife enden.

`geheimzahl = 7`

Schreibe hier deine Schleife

Dein Code:

Projekt: Quiz-Programm

Im letzten Projekt baust du ein kleines **Quiz-Programm**. Es nutzt eine **Funktion mit Rückgabewert**, um Fragen zu stellen und zu prüfen, ob die Antwort stimmt.

Eine Funktion, die eine Frage stellt und True/False zurückgibt:

Beispiel

```
def frage_stellen(frage, richtige_antwort):  
    antwort = input(frage)  
    return antwort == richtige_antwort
```

`frage_stellen(...)` liefert True oder False zurück – das kannst du direkt in einem `if` benutzen, um einen Zähler zu erhöhen: `if frage_stellen(...): richtig = richtig + 1`.

Aufgabe 3: Warum gibt `frage_stellen()` das Ergebnis mit `return` statt mit `print()` zurück?

- ☐ a) Damit man das Ergebnis danach in einem `if` weiterverwenden kann
- ☐ b) `print()` funktioniert in Funktionen generell nicht
- ☐ c) Das ist egal, beides wäre identisch



Aufgabe 4: Vervollständige die Funktion `frage_stellen(frage, richtige_antwort)`: Sie soll die Frage per `input()` stellen und `True` oder `False` zurückgeben (`Antwort == richtige_antwort`). Rufe sie danach zweimal auf – mit "Hauptstadt von Frankreich? " / "Paris" und mit "Hauptstadt von Italien? " / "Rom" – und zähle die richtigen Antworten in `richtig`. Gib am Ende Du hast X von 2 richtig! aus.

```
def frage_stellen(frage, richtige_antwort):  
    # Hier deine Lösung  
    pass  
  
richtig = 0  
  
# Rufe frage_stellen() zweimal auf und zähle richtig  
  
Dein Code:
```



🎉 **Geschafft!** Du hast den kompletten Grundkurs durchgearbeitet – von der ersten Ausgabe mit `print()` bis zu eigenen Funktionen, Klassen und einem echten kleinen Programm. Du kannst jetzt eigene Ideen in Python umsetzen. Weiter so!

Projekt: Vokabeltrainer

Zum Abschluss baust du einen **Vokabeltrainer** – und dafür brauchst du fast alles, was du im ganzen Kurs gelernt hast:

ein **Dictionary** für die Vokabeln, eine **for-Schleife** zum Durchgehen, eine **Funktion** zum Prüfen, eine **Bedingung** für richtig oder falsch, eine **Liste** für die Fehler und zum Schluss eine **Datei**, in der du sie speicherst.

Genau darum geht es beim Programmieren: nicht ein Werkzeug, sondern das richtige Werkzeug zur richtigen Zeit.

Aufgabe 5: Die Vokabeln stehen schon als Dictionary bereit. Gib jedes Paar in einer eigenen Zeile aus, im Format Hund = dog.

Nutze dafür eine for-Schleife über das Dictionary.

```
vokabeln = {"Hund": "dog", "Katze": "cat", "Maus": "mouse"}
```

```
# Gib jedes Paar aus
```

Dein Code:



Beim Abfragen tippt niemand exakt so, wie es im Dictionary steht: mal groß, mal klein, mal mit einem Leerzeichen am Ende. Ein stures == würde "Dog" als falsch werten.

Deshalb kommt die Prüfung in eine eigene **Funktion** – und dort räumst du die Eingabe erst auf, bevor du vergleichst. Dafür brauchst du zwei String-Methoden aus Kapitel 4: `.lower()` macht alles klein, `.strip()` entfernt Leerzeichen am Rand.

Aufgabe 6: Schreibe eine Funktion `ist_richtig(eingabe, loesung)`. Sie soll `True` zurückgeben, wenn die Eingabe der Lösung entspricht – und dabei Groß- und Kleinschreibung sowie Leerzeichen am Rand ignorieren.

Rufe sie danach dreimal auf und gib das Ergebnis aus:

`ist_richtig("DOG", "dog")`, `ist_richtig(" cat ", "cat")` und `ist_richtig("maus", "mouse")`.

```
def ist_richtig(eingabe, loesung):  
    # Hier deine Lösung  
    pass
```

Rufe die Funktion dreimal auf und gib das Ergebnis aus

Dein Code:



Jetzt das Beste am Üben: Was man **falsch** hatte, sollte man sich merken. Du sammelst die falschen Vokabeln in einer **Liste** und schreibst sie am Ende in eine **Datei** – dann sind sie auch nach dem Schließen des Programms noch da.

Aufgabe 7: Die Vokabeln und die (teils falschen) Antworten stehen bereit. Gehe alle Vokabeln durch und sammle die deutschen Wörter, bei denen die Antwort nicht stimmt, in der Liste falsche.

Schreibe die Liste danach in die Datei **falsche.txt** – ein Wort pro Zeile – lies die Datei wieder ein und gib ihren Inhalt aus.

```
vokabeln = {"Hund": "dog", "Katze": "cat", "Maus": "mouse"}  
antworten = {"Hund": "dog", "Katze": "dog", "Maus": "mice"}  
falsche = []
```

```
# Sammle die falschen, speichere sie und gib die Datei aus
```

Dein Code:



Bisher liegen die Vokabeln, die Prüffunktion und der Fehlerzähler lose nebeneinander. Dabei gehören sie zusammen – und genau dafür gibt es **Klassen** aus Kapitel 14.

In einer Klasse steckt beides in einem: die **Daten** (die Vokabeln, die Zahl der Fehler) als Attribute und das **Können** (prüfen) als Methode. Der Trainer wird damit zu einem einzigen Ding, das man herumreichen kann.

Aufgabe 8: Wiederholung aus Kapitel 14: Schreibe eine Klasse `Vokabeltrainer`.

`__init__(self, vokabeln)` speichert das Dictionary in `self.vokabeln` und setzt `self.fehler` auf 0.

Die Methode `pruefe(self, deutsch, antwort)` gibt `True` zurück, wenn die Antwort stimmt – wieder **ohne** auf Groß- und Kleinschreibung zu achten. Stimmt sie nicht, zählt sie `self.fehler` um eins hoch und gibt `False` zurück.

Erzeuge damit einen Trainer für `{"Hund": "dog", "Katze": "cat"}`, prüfe "Hund" mit "DOG" und "Katze" mit "dog", gib beide Ergebnisse aus und danach die Zahl der Fehler.

```
class Vokabeltrainer:
    def __init__(self, vokabeln):
        # Hier deine Lösung
        pass

    def pruefe(self, deutsch, antwort):
        # Hier deine Lösung
        pass

# Erzeuge einen Trainer und prüfe zwei Vokabeln

Dein Code:
```

Geschafft. In diesem einen Programm steckt fast der ganze Kurs: Dictionary, Schleife, Funktion, Bedingung, Liste, String-Methoden, Datei und zuletzt eine eigene Klasse.

Genau so sieht echte Programmierarbeit aus – man kombiniert, was man kann, statt ein einzelnes Werkzeug zu suchen.